

execution. The event loop continues running in the background and can run other non-blocked tasks.

Remember that *await* can only be called inside a coroutine defined using *async def*.

A sample program code is given below:

```
import asyncio

async def myfun(name, delay):
    print(f"Task {name} started")
    await asyncio.sleep(delay)
    print(f"Task {name} finished")

async def main():
    await asyncio.gather(
        myfun("A", 2),
        myfun("B", 1),
        myfun("C", 3)
    )

asyncio.run(main())
```

Output

```
Task A started
Task B started
Task C started
Task B finished
Task A finished
Task C finished
```

Here's an explanation of the code.

```
import asyncio
```

asyncio is Python's built-in library for writing asynchronous, non-blocking programs. It provides tools like:

- Event loops
- Coroutines
- Tasks
- Asynchronous I/O helpers

```
async def myfun(name, delay)
```

The *async* keyword makes *myfun* a coroutine. It can use *await* inside it and run concurrently with other coroutines.

```
await asyncio.sleep(delay)
```

This is the key part. *asyncio.sleep()* is non-blocking. It tells the event loop 'pause this coroutine for delay seconds'. While this coroutine waits, the event loop can run other tasks. No CPU is wasted, and the program does not freeze.

This means:

- Task A waits 2 seconds
- Task B waits 1 second
- Task C waits 3 seconds

But all three tasks wait concurrently. *asyncio.gather* schedules all three coroutines to run at the same time.

The timeline becomes:

- Time 0s → Start A, B, C
- Time 1s → B finishes
- Time 2s → A finishes
- Time 3s → C finishes

So the total runtime is 3 seconds and not $2 + 1 + 3 = 6$ seconds.

```
asyncio.run(main())
```

This creates an event loop, runs the *main()* coroutine, waits until all tasks inside it complete, and closes the event loop.

This is how the event loop works:

- Picks a coroutine ready to run.
- Runs it until it explicitly calls *await*.
- Pauses it and stores its state.
- Switches to another ready coroutine.
- Gets back to a blocked coroutine when it becomes ready again.

The important question is: what does the event loop do when a previously blocked coroutine is ready, but another coroutine is already running? Well, the event loop never interrupts or pre-empt a running coroutine. Python *async* equals cooperative multitasking, and not pre-emptive multitasking. So, the event loop will let the currently running coroutine continue until it reaches an *await* (or returns). Only then will it switch and resume the coroutine that just became ready.

A coroutine must voluntarily yield the CPU by calling one of the following:

- *Await <someio>*
- *Await asyncio.sleep*
- Return

Then only event loop can switch — there is no pre-emption. So, if a coroutine never awaits, the event loop never switches to another task. Therefore, you must never write blocking code inside a coroutine.

Happy asynchronous programming in Python! 

 By: Supriyo Ganguly

The author is an M. Tech in embedded systems and works as a senior technical officer at the Electronics Corporation of India Limited, Hyderabad.